Lunix

BASH

Karim Madani

🧠 1️⃣ Définition simple (niveau débutant)

👉 **Bash** = un langage + un interpréteur de commandes Linux

- Bash = **Bourne Again SHell**
- C'est ce que tu utilises quand tu écris :

```
ls
cd
pwd
```

👉 Donc :

- Terminal = interface
- Bash = cerveau qui exécute les commandes

🎯 Analogie

👉 Bash = chef d'orchestre 🎼
👉 Commandes = musiciens
👉 Script = partition complète

⚙️ 2️⃣ Principe de fonctionnement

Quand tu écris :

```
ls
```

👉 Bash fait :

1. lit la commande
2. cherche le programme `/bin/ls`
3. exécute
4. affiche résultat

🔎 Version avancée

👉 Bash est un **interpréteur de commandes POSIX**

👉 Il gère :

- variables
- boucles
- conditions
- processus

🧩 3️⃣ Structure d'un script Bash

📌 Exemple minimal

```
#!/bin/bash

echo "Hello World"
```

🔍 Explication

◇ `#!/bin/bash`

👉 appelé **shebang**

👉 signifie :

"exécuter ce script avec Bash"

◇ `echo`

👉 afficher du texte

▶️ Exécuter un script

Étapes :

```
chmod +x script.sh
./script.sh
```

🧠 4️⃣ Variables (TRÈS IMPORTANT)

📌 Définition simple

👉 Variable = boîte qui stocke une valeur

📌 Exemple

```
name="karim"
echo $name
```

⚠️ Attention

❌ FAUX :

```
name = "karim"
```

✔️ CORRECT :

```
name="karim"
```

💧 Variables système

```
echo $HOME
echo $USER
echo $PATH
```

🧠 Version avancée

👉 Bash est **faiblement typé**

- pas besoin de déclarer type (int, string)

🔄 5️⃣ Conditions (if)

📌 Exemple simple

```
#!/bin/bash

if [ "$name" = "karim" ]
then
    echo "Welcome"
fi
```

🔍 Explication

- `[ condition ]` = test
- `then` = action
- `fi` = fin

💧 Comparaisons

Test	Signification
`=`	égal
`!=`	différent
`-eq`	égal (nombre)
`-gt`	plus grand
`-lt`	plus petit

🔁 6️⃣ Boucles (loops)

◇ for

```
for i in 1 2 3
do
    echo $i
done
```

◇ while

```
count=1

while [ $count -le 3 ]
do
    echo $count
    ((count++))
done
```

🧠 Vision hacker

👉 utilisé pour :

- brute force

- scan automatique
- test de vulnérabilités

7 Input utilisateur

```
echo "Enter name:"
read name

echo "Hello $name"
```

👉 Bash peut interagir avec utilisateur

8 Manipulation fichiers

vérifier fichier

```
if [ -f file.txt ]
then
    echo "exists"
fi
```

types

Test	Signification
-f	fichier
-d	dossier
-r	lisible
-w	modifiable
-x	exécutable

🔗 9️⃣ Pipes & redirection

📌 pipe

```
ls | grep txt
```

📌 redirection

```
echo "hello" > file.txt
echo "world" >> file.txt
```

🧠 Vision hacker

👉 permet :

- filtrer données
- extraire infos sensibles

⚡ 🔟 Command substitution

```
date_now=$(date)
echo $date_now
```

👉 exécuter commande dans variable

🔐 1️⃣ Version cybersécurité

🎯 Exemple script hacker simple

```
#!/bin/bash

echo "Scanning system..."

for file in /home/*
do
    if [ -w "$file" ]
    then
        echo "Writable: $file"
    fi
done
```

👉 trouve fichiers modifiables = vulnérabilité

🧪 1️⃣2️⃣ Exercice pratique

🎯 Objectif :

Créer script qui :

1. demande un nom
2. crée un fichier
3. affiche contenu

💡 Solution

```
#!/bin/bash

echo "Enter filename:"
read file

touch $file

echo "File created: $file"
```

🧠 1 3 Niveau avancé (important)

🔥 Bash = langage de scripting système

👉 utilisé pour :

- automatisation
- pentesting
- administration système

⚠️ erreurs fréquentes

❌ oublier $
❌ espaces dans variables
❌ oublier `chmod +x`

📌 1 4 Résumé rapide

- Bash = langage Linux
- echo → afficher

- variables → `$name`
- `if` → condition
- `for` / `while` → boucles
- `read` → input
- `|` → pipe
- `>` → redirection

🧠 🧩 Mental Model

Pense toujours :

👉 "Comment automatiser cette tâche ?"